push-swap

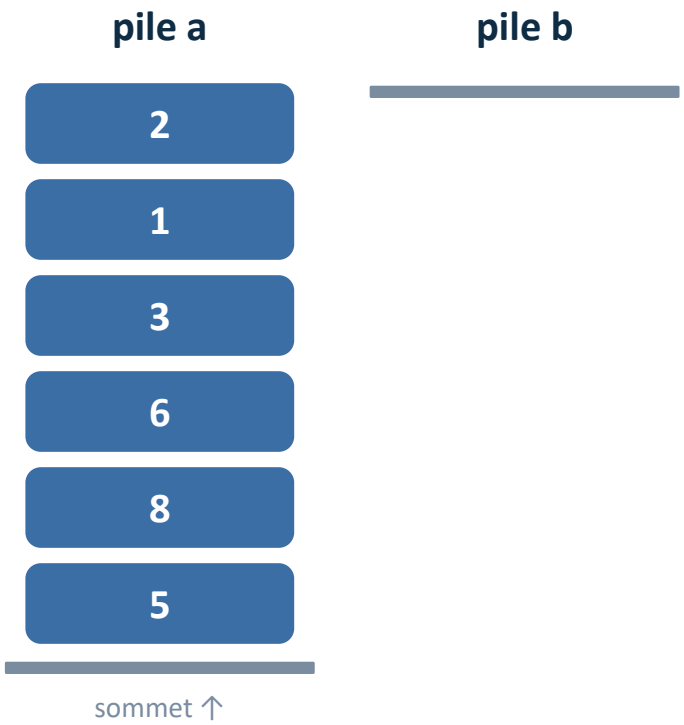
Trier une pile avec le moins de coups possible

Projet en Go • 2 programmes • 2 piles • 11 instructions

L'objectif

Trier la pile a par ordre croissant, avec le plus petit nombre au sommet, en utilisant uniquement une seconde pile b et un jeu d'instructions limité.

Le défi : trouver la plus courte suite d'instructions possible.



Deux programmes à écrire

`push-swap`

Calcule et affiche la plus courte liste d'instructions qui trie la pile

- Entrée : liste d'entiers
- Sortie : une instruction par ligne
- Objectif : minimiser le nombre de coups

`checker`

Lit des instructions, les exécute et vérifie le résultat

- Entrée : entiers + instructions (stdin)
- Sortie : OK ou KO
- OK = a triée et b vide

Les 11 instructions

Pousser

pa b → a

pb a → b

Échanger

sa 2 premiers de a

sb 2 premiers de b

ss sa + sb

Rotation ↑

ra a : 1er devient dernier

rb b : idem

rr ra + rb

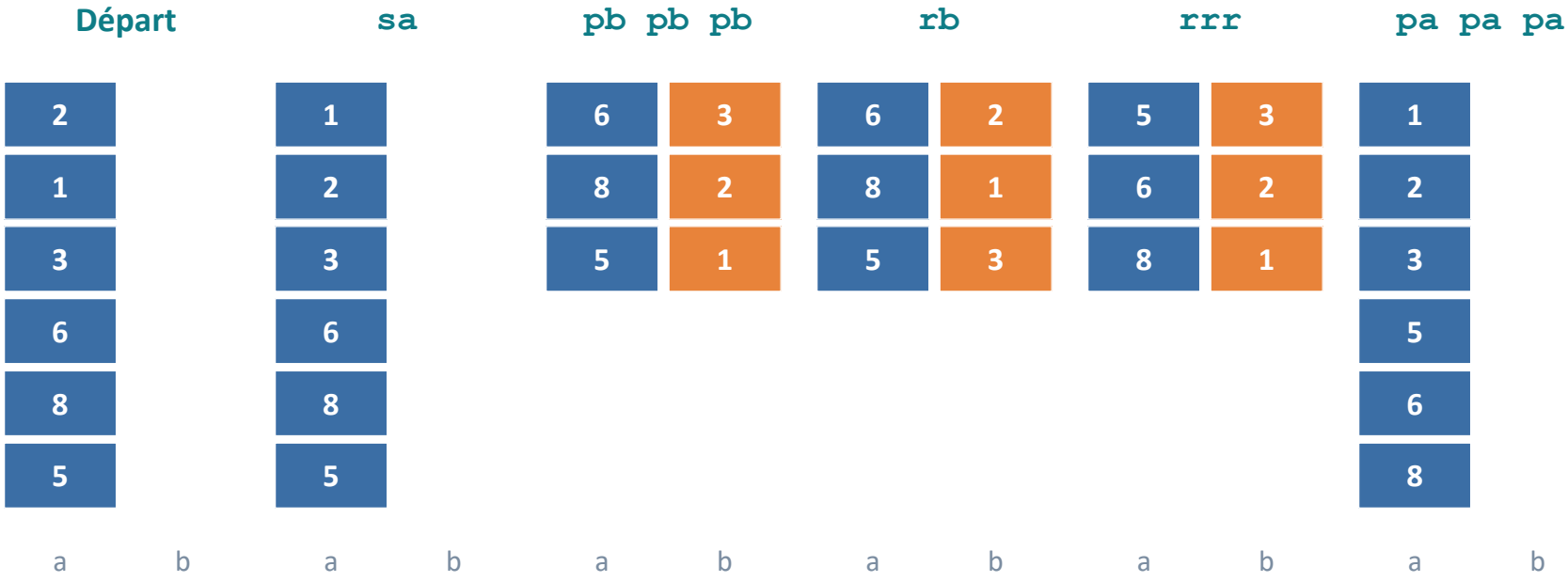
Rotation ↓

rra a : dernier devient 1er

rrb b : idem

rrr rra + rrb

Exemple pas à pas



Cet exemple est celui du sujet : on termine avec a triée et b vide.

Utilisation et gestion des erreurs

```
$ ARG="4 67 3 87 23"
$ ./push-swap "$ARG" | wc -l
6
$ ./push-swap "$ARG" | ./checker "$ARG"
OK
```

Cas d'erreur → Error sur la sortie d'erreur standard

- argument qui n'est pas un entier
- doublon dans la liste
- instruction inexistante ou mal formatée (checker)
- aucun argument : le programme n'affiche rien

Contraintes et compétences visées

Contraintes

- Écrit en Go
- Uniquement la bibliothèque standard
- Exécutables nommés checker et push-swap
- Bonnes pratiques et tests unitaires conseillés
- Gestion des erreurs obligatoire

Vous allez apprendre

- Les algorithmes de base
- Les algorithmes de tri
- L'utilisation des piles
- À optimiser : moins d'instructions = meilleur résultat

À vous de jouer

1. Modéliser les piles et les 11 instructions
2. Écrire checker pour valider
3. Concevoir l'algorithme de tri de push-swap
4. Mesurer et réduire le nombre d'instructions